

MSSQL, Redis, HTTP ve Gateway Performans İncelemesi

Özet

Performans değişiklikleri financial correctness'i zayıflatmadan uygulanmalıdır. Operasyonel tuning değerleri mümkün olduğunda configuration-driven'dir; accounting/locking invariant'ları benchmark uğruna gevşetilmez.

MSSQL connection pooling

SqlConnection pooling connection string ile açıktır. Development baseline:

- Pooling=True;
- Min Pool Size=5;
- Max Pool Size=100;
- bounded connect timeout;
- Load Balance Timeout;
- application name.

SqlConnectionFactory short-lived logical connection üretir; dispose fiziksel connection'ı pool'a döndürür. Production pool size ölçülmüş DB kapasitesine göre ayarlanmalıdır.

Financial transaction performansı

Wallet transfer aynı transaction içinde source/destination balances, idempotency, FinancialTransaction ve Ledger'ı commit eder. Lock duration önemlidir ama correctness'ten daha önemli değildir. Wallet row'ları deterministic GUID order ile lock edilir; opposite-direction deadlock riskini azaltır.

Isolation level yalnız throughput artırmak için zayıflatılmaz.

Index yaklaşımı

Mevcut unique/index yapıları hot path'lerin önemli bölümünü destekler. Financial write-heavy tablolara speculative index eklemek insert/update/log maliyetini artırabilir. Yeni index Query Store, execution plan, logical read ve measured p95/p99 kanıtına göre eklenmelidir.

İzlenecek DB metrikleri:

- pool exhaustion;
- login/transfer DB p95/p99;
- deadlock;
- lock wait;
- top logical reads;
- Query Store regression;
- transaction-log growth;
- index usage/fragmentation.

Redis

Tek ConnectionMultiplexer singleton tutulur. Per-request multiplexer socket/performance regression olur.

Configurable değerler:

- AbortOnConnectFail;
- connect retry;
- connect timeout;
- sync timeout;
- keep-alive;
- client name.

OTP security fail-closed davranışı performans için gevşetilmez. Redis financial authority değildir. İzlenecekler: reconnect, latency, timeout, CPU, memory, eviction, network, Lua latency.

HttpClient

Provider client'ları HttpClientFactory + SocketsHttpHandler kullanır:

- pooled connection lifetime;
- pooled idle timeout;
- max connections/server;
- GZip/Deflate/Brotli;
- provider-specific timeout;
- cookies disabled.

Provider çağrıları YARP Gateway'e gider. Financial POST'lara endpoint-specific idempotency kanıtı olmadan otomatik retry eklenmez.

YARP

Clusterlar PowerOfTwoChoices kullanır. Production destination replica sayısı config ile artırılabilir. Performance controls:

- load balancing;
- active health;
- main cluster passive health;
- max destination connections;
- activity timeout;
- HTTP version policy;
- response buffering policy;
- pre-proxy request-size limit.

Rate limiting

Gateway public limit backend'den daha sıkıdır. Backend ikinci katman bypass/misrouting/internal runaway korumasıdır. Queue default 0; overload sırasında request biriktirmek yerine 429 ile hızlı reddetmek tail latency/memory baskısını azaltır.

Bilinçli yapılmayan optimizasyonlar

- Wallet balance'ı Redis source of truth yapmak yok.
- Financial SQL isolation'ı körlemesine düşürmek yok.
- Arbitrary financial POST retry yok.
- Authenticated financial response caching yok.

- Unbounded DB/HTTP connection pool yok.
- Fraud check'i latency için bypass etmek yok.
- Reconciliation'ı bozacak ledger denormalization yok.

Benchmark planı

Ölç:

- 1 registration RPS;
- 2 login p50/p95/p99;
- 3 wallet-list p50/p95/p99;
- 4 transfer p50/p95/p99;
- 5 same-wallet concurrent transfer throughput;
- 6 Gateway overhead vs controlled direct internal baseline;
- 7 Redis OTP latency;
- 8 SQL locks/deadlocks;
- 9 CPU/memory/GC/socket;
- 10 load sonrası ledger/balance/idempotency reconciliation.

Final balances/ledger reconcile etmeyen benchmark sonucu geçersizdir.